



Red Hat OpenShift AI Self-Managed 3.0

Configuring your model-serving platform

Configure your model-serving platform in Red Hat OpenShift AI Self-Managed

Red Hat OpenShift AI Self-Managed 3.0 Configuring your model-serving platform

Configure your model-serving platform in Red Hat OpenShift AI Self-Managed

Legal Notice

Copyright © 2025 Red Hat, Inc.

The text of and illustrations in this document are licensed by Red Hat under a Creative Commons Attribution–Share Alike 3.0 Unported license ("CC-BY-SA"). An explanation of CC-BY-SA is available at

<http://creativecommons.org/licenses/by-sa/3.0/>

. In accordance with CC-BY-SA, if you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, Red Hat Enterprise Linux, the Shadowman logo, the Red Hat logo, JBoss, OpenShift, Fedora, the Infinity logo, and RHCE are trademarks of Red Hat, Inc., registered in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

Java[®] is a registered trademark of Oracle and/or its affiliates.

XFS[®] is a trademark of Silicon Graphics International Corp. or its subsidiaries in the United States and/or other countries.

MySQL[®] is a registered trademark of MySQL AB in the United States, the European Union and other countries.

Node.js[®] is an official trademark of Joyent. Red Hat is not formally related to or endorsed by the official Joyent Node.js open source or commercial project.

The OpenStack[®] Word Mark and OpenStack logo are either registered trademarks/service marks or trademarks/service marks of the OpenStack Foundation, in the United States and other countries and are used with the OpenStack Foundation's permission. We are not affiliated with, endorsed or sponsored by the OpenStack Foundation, or the OpenStack community.

All other trademarks are the property of their respective owners.

Abstract

As a Red Hat OpenShift AI administrator, you can configure your model serving platform in Red Hat OpenShift AI Self-Managed.

Table of Contents

CHAPTER 1. ABOUT MODEL-SERVING PLATFORMS	3
1.1. ABOUT MODEL SERVING	3
1.1.1. Model serving platform	3
1.1.2. NVIDIA NIM model serving platform	3
1.2. MODEL-SERVING RUNTIMES	4
1.2.1. ServingRuntime	4
1.2.2. InferenceService	5
1.3. MODEL-SERVING RUNTIMES FOR ACCELERATORS	6
1.3.1. NVIDIA GPUs	6
1.3.2. Intel Gaudi accelerators	6
1.3.3. AMD GPUs	7
1.3.4. IBM Spyre AI accelerators on x86 and IBM Z	7
1.3.5. Supported model-serving runtimes	8
1.3.6. Tested and verified model-serving runtimes	8
CHAPTER 2. CONFIGURING MODEL SERVERS	9
2.1. ENABLING THE MODEL SERVING PLATFORM	9
2.2. ENABLING SPECULATIVE DECODING AND MULTI-MODAL INFERENCING	9
2.3. ADDING A CUSTOM MODEL-SERVING RUNTIME	11
2.4. ADDING A TESTED AND VERIFIED RUNTIME	13
CHAPTER 3. CONFIGURING MODEL SERVERS ON THE NVIDIA NIM MODEL SERVING PLATFORM	23
3.1. ENABLING THE NVIDIA NIM MODEL SERVING PLATFORM	23
CHAPTER 4. CUSTOMIZING MODEL DEPLOYMENTS	25
4.1. CUSTOMIZING THE PARAMETERS OF A DEPLOYED MODEL-SERVING RUNTIME	25
4.2. CUSTOMIZABLE MODEL SERVING RUNTIME PARAMETERS	26
4.3. CUSTOMIZING THE VLLM MODEL-SERVING RUNTIME	26

CHAPTER 1. ABOUT MODEL-SERVING PLATFORMS

As an OpenShift AI administrator, you can enable your preferred serving platform and make it available for serving models. You can also add a custom or a tested and verified model-serving runtime.

1.1. ABOUT MODEL SERVING

When you serve a model, you upload a trained model into Red Hat OpenShift AI for querying, which allows you to integrate your trained models into intelligent applications.

You can upload a model to an S3-compatible object storage, persistent volume claim, or Open Container Initiative (OCI) image. You can then access and train the model from your project workbench. After training the model, you can serve or deploy the model using a model-serving platform.

Serving or deploying the model makes the model available as a service, or model runtime server, that you can access using an API. You can then access the inference endpoints for the deployed model from the dashboard and see predictions based on data inputs that you provide through API calls. Querying the model through the API is also called model inferencing.

You can also serve models on the NVIDIA NIM model serving platform. The model-serving platform that you choose depends on your business needs:

- If you want to deploy each model on its own runtime server, select the **Model serving platform**. The model serving platform is recommended for production use.
- If you want to use NVIDIA Inference Microservices (NIMs) to deploy a model, select the **NVIDIA NIM-model serving platform**.

1.1.1. Model serving platform

You can deploy each model from a dedicated model server, which can help you deploy, monitor, scale, and maintain models that require increased resources. Based on the [KServe](#) component, this model serving platform is ideal for serving large models.

The model serving platform is helpful for use cases such as:

- Large language models (LLMs)
- Generative AI

For more information about setting up the model serving platform, see [Installing the model serving platform](#).

1.1.2. NVIDIA NIM model serving platform

You can deploy models using NVIDIA Inference Microservices (NIM) on the NVIDIA NIM model serving platform.

NVIDIA NIM, part of NVIDIA AI Enterprise, is a set of microservices designed for secure, reliable deployment of high performance AI model inferencing across clouds, data centers and workstations.

NVIDIA NIM inference services are helpful for use cases such as:

- Using GPU-accelerated containers inferencing models optimized by NVIDIA

- Deploying generative AI for virtual screening, content generation, and avatar creation

The NVIDIA NIM model serving platform is based on the model serving platform. To use the NVIDIA NIM model serving platform, you must first install the model serving platform.

For more information, see [Installing the model serving platform](#).

1.2. MODEL-SERVING RUNTIMES

You can serve models on the model serving platform by using model-serving runtimes. The configuration of a model-serving runtime is defined by the **ServingRuntime** and **InferenceService** custom resource definitions (CRDs).

1.2.1. ServingRuntime

The **ServingRuntime** CRD creates a serving runtime, an environment for deploying and managing a model. It creates the templates for pods that dynamically load and unload models of various formats and also exposes a service endpoint for inferencing requests.

The following YAML configuration is an example of the **vLLM ServingRuntime for KServe** model-serving runtime. The configuration includes various flags, environment variables and command-line arguments.

```
apiVersion: serving.kserve.io/v1alpha1
kind: ServingRuntime
metadata:
  annotations:
    opendatahub.io/recommended-accelerators: ["nvidia.com/gpu"] 1
    openshift.io/display-name: vLLM ServingRuntime for KServe 2
  labels:
    opendatahub.io/dashboard: "true"
  name: vllm-runtime
  namespace: <namespace>
spec:
  annotations:
    prometheus.io/path: /metrics 3
    prometheus.io/port: "8080" 4
  containers:
    - args:
        - --port=8080
        - --model=/mnt/models 5
        - --served-model-name={{.Name}} 6
      command: 7
        - python
        - '-m'
        - vllm.entrypoints.openai.api_server
      env:
        - name: HF_HOME
          value: /tmp/hf_home
      image:
        quay.io/modh/vllm@sha256:8a3dd8ad6e15fe7b8e5e471037519719d4d8ad3db9d69389f2beded36a6f5
        b21 8
      name: kserve-container
      ports:
```



```

- containerPort: 8080
  protocol: TCP
multiModel: false 9
supportedModelFormats: 10
- autoSelect: true
  name: vLLM

```

- ¹ The recommended accelerator to use with the runtime.
- ² The name with which the serving runtime is displayed.
- ³ The endpoint used by Prometheus to scrape metrics for monitoring.
- ⁴ The port used by Prometheus to scrape metrics for monitoring.
- ⁵ The path to where the model files are stored in the runtime container.
- ⁶ Passes the model name that is specified by the `{{.Name}}` template variable inside the runtime container specification to the runtime environment. The `{{.Name}}` variable maps to the `spec.predictor.name` field in the **InferenceService** metadata object.
- ⁷ The entrypoint command that starts the runtime container.
- ⁸ The runtime container image used by the serving runtime. This image differs depending on the type of accelerator used.
- ⁹ Specifies that the runtime is used for model serving.
- ¹⁰ Specifies the model formats supported by the runtime.

1.2.2. InferenceService

The **InferenceService** CRD creates a server or inference service that processes inference queries, passes it to the model, and then returns the inference output.

The inference service also performs the following actions:

- Specifies the location and format of the model.
- Specifies the serving runtime used to serve the model.
- Enables the passthrough route for gRPC or REST inference.
- Defines HTTP or gRPC endpoints for the deployed model.

The following example shows the InferenceService YAML configuration file that is generated when deploying a granite model with the vLLM runtime:

```

apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  annotations:
    openshift.io/display-name: granite
    serving.knative.openshift.io/enablePassthrough: 'true'
    sidecar.istio.io/inject: 'true'

```

```

    sidecar.istio.io/rewriteAppHTTPProbers: 'true'
name: granite
labels:
  opendatahub.io/dashboard: 'true'
spec:
  predictor:
    maxReplicas: 1
    minReplicas: 1
    model:
      modelFormat:
        name: vLLM
      name: ""
    resources:
      limits:
        cpu: '6'
        memory: 24Gi
        nvidia.com/gpu: '1'
      requests:
        cpu: '1'
        memory: 8Gi
        nvidia.com/gpu: '1'
    runtime: vllm-runtime
    storage:
      key: aws-connection-my-storage
      path: models/granite-7b-instruct/
  tolerations:
    - effect: NoSchedule
      key: nvidia.com/gpu
      operator: Exists

```

Additional resources

- [KServe Serving Runtimes](#)

1.3. MODEL-SERVING RUNTIMES FOR ACCELERATORS

OpenShift AI provides support for accelerators through preinstalled model-serving runtimes.

1.3.1. NVIDIA GPUs

You can serve models with NVIDIA graphics processing units (GPUs) by using the **vLLM NVIDIA GPU ServingRuntime for KServe** runtime. To use the runtime, you must enable GPU support in OpenShift AI. This includes installing and configuring the Node Feature Discovery Operator on your cluster. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).

1.3.2. Intel Gaudi accelerators

You can serve models with Intel Gaudi accelerators by using the **vLLM Intel Gaudi Accelerator ServingRuntime for KServe** runtime. To use the runtime, you must enable hybrid processing (HPU) support in OpenShift AI. This includes installing the Intel Gaudi Base Operator and configuring a hardware profile. For more information, see [Intel Gaudi Base Operator OpenShift installation](#) and [Working with hardware profiles](#).

For information about recommended vLLM parameters, environment variables, supported configurations and more, see [vLLM with Intel® Gaudi® AI Accelerators](#).



NOTE

Warm-up is a model initialization and performance optimization step that is useful for reducing cold-start delays and first-inference latency. Depending on the model size, warm-up can lead to longer model loading times.

While highly recommended in production environments to avoid performance limitations, you can choose to skip warm-up for non-production environments to reduce model loading times and accelerate model development and testing cycles. To skip warm-up, follow the steps described in [Customizing the parameters of a deployed model-serving runtime](#) to add the following environment variable in the **Configuration parameters** section of your model deployment:

```
`VLLM_SKIP_WARMUP="true"
```

1.3.3. AMD GPUs

You can serve models with AMD GPUs by using the **vLLM AMD GPU ServingRuntime for KServe** runtime. To use the runtime, you must enable support for AMD graphic processing units (GPUs) in OpenShift AI. This includes installing the AMD GPU operator and configuring a hardware profile. For more information, see [Deploying the AMD GPU operator on OpenShift](#) in the AMD documentation and [Working with hardware profiles](#).

1.3.4. IBM Spyre AI accelerators on x86 and IBM Z



IMPORTANT

Support for IBM Spyre AI Accelerators on x86 is currently available in Red Hat OpenShift AI 3.0 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Support for IBM Spyre AI Accelerators on s390x is currently available in Red Hat OpenShift AI 3.0 as a General Availability (GA) feature.

You can serve models with IBM Spyre AI accelerators on x86 by using the **vLLM Spyre AI Accelerator ServingRuntime for KServe** runtime. For IBM Z (s390x architecture), use the **vLLM Spyre s390x ServingRuntime for KServe** runtime. To use the runtime, you must install the Spyre Operator and configure a hardware profile. For more information, see [Spyre operator image](#) and [Working with hardware profiles](#).

Additional resources

- [Supported model-serving runtimes](#)

1.3.5. Supported model-serving runtimes

OpenShift AI includes several preinstalled model-serving runtimes. You can use preinstalled model-serving runtimes to start serving models without modifying or defining the runtime yourself. You can also add a custom runtime to support a model.

See [Supported Configurations for 3.x](#) for a list of the supported model-serving runtimes and deployment requirements.

For help adding a custom runtime, see [Adding a custom model-serving runtime](#).

Additional resources

- [Inference endpoints](#)

1.3.6. Tested and verified model-serving runtimes

Tested and verified runtimes are community versions of model-serving runtimes that have been tested and verified against specific versions of OpenShift AI.

Red Hat tests the current version of a tested and verified runtime each time there is a new version of OpenShift AI. If a new version of a tested and verified runtime is released in the middle of an OpenShift AI release cycle, it will be tested and verified in an upcoming release.

See [Supported Configurations for 3.x](#) for a list of tested and verified runtimes in OpenShift AI.



NOTE

Tested and verified runtimes are not directly supported by Red Hat. You are responsible for ensuring that you are licensed to use any tested and verified runtimes that you add, and for correctly configuring and maintaining them.

For more information, see [Tested and verified runtimes in OpenShift AI](#).

Additional resources

- [Inference endpoints](#)

CHAPTER 2. CONFIGURING MODEL SERVERS

You configure model servers by using model-serving runtimes, which add support for a specified set of model frameworks and the model formats that they support.

2.1. ENABLING THE MODEL SERVING PLATFORM

When you have installed KServe, you can use the Red Hat OpenShift AI dashboard to enable the model serving platform. You can also use the dashboard to enable model-serving runtimes for the platform.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- You have installed KServe.
- The **spec.dashboardConfig.disableKServe** dashboard configuration option is set to **false** (the default).
For more information about setting dashboard configuration options, see [Customizing the dashboard](#).

Procedure

1. Enable the model serving platform as follows:
 - a. In the left menu, click **Settings** → **Cluster settings** → **General settings**.
 - b. Locate the **Model serving platforms** section.
 - c. To enable the model serving platform for projects, select the **Model serving platform** checkbox.
 - d. Click **Save changes**.
2. Enable preinstalled runtimes for the model serving platform as follows:
 - a. In the left menu of the OpenShift AI dashboard, click **Settings** → **Model resources and operations** → **Serving runtimes**.
The **Serving runtimes** page shows preinstalled runtimes and any custom runtimes that you have added.

For more information about preinstalled runtimes, see [Supported runtimes](#).
 - b. Set the runtime that you want to use to **Enabled**.
The model serving platform is now available for model deployments.

2.2. ENABLING SPECULATIVE DECODING AND MULTI-MODAL INFERENCE

You can configure the **vLLM NVIDIA GPU ServingRuntime for KServer** runtime to use speculative decoding, a parallel processing technique to optimize inferencing time for large language models (LLMs).

You can also configure the runtime to support inferencing for vision-language models (VLMs). VLMs are a subset of multi-modal models that integrate both visual and textual data.

The following procedure describes customizing the **vLLM NVIDIA GPU ServingRuntime for KServe** runtime for speculative decoding and multi-modal inferencing.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- If you are using the vLLM model-serving runtime for speculative decoding with a draft model, you have stored the original model and the speculative model in the same folder within your S3-compatible object storage.

Procedure

1. Follow the steps to deploy a model as described in [Deploying models on the model serving platform](#).
2. In the **Serving runtime** field, select the **vLLM NVIDIA GPU ServingRuntime for KServe** runtime.
3. To configure the vLLM model-serving runtime for speculative decoding by matching n-grams in the prompt, add the following arguments under **Additional serving runtime arguments** in the **Configuration parameters** section:

```
--speculative-model=[ngram]
--num-speculative-tokens=<NUM_SPECULATIVE_TOKENS>
--ngram-prompt-lookup-max=<NGRAM_PROMPT_LOOKUP_MAX>
--use-v2-block-manager
```

- a. Replace **<NUM_SPECULATIVE_TOKENS>** and **<NGRAM_PROMPT_LOOKUP_MAX>** with your own values.



NOTE

Inferencing throughput varies depending on the model used for speculating with n-grams.

4. To configure the vLLM model-serving runtime for speculative decoding with a draft model, add the following arguments under **Additional serving runtime arguments** in the **Configuration parameters** section:

```
--port=8080
--served-model-name={{.Name}}
--distributed-executor-backend=mp
--model=/mnt/models/<path_to_original_model>
--speculative-model=/mnt/models/<path_to_speculative_model>
--num-speculative-tokens=<NUM_SPECULATIVE_TOKENS>
--use-v2-block-manager
```

- a. Replace **<path_to_speculative_model>** and **<path_to_original_model>** with the paths to the speculative model and original model on your S3-compatible object storage.
- b. Replace **<NUM_SPECULATIVE_TOKENS>** with your own value.

5. To configure the vLLM model-serving runtime for multi-modal inferencing, add the following arguments under **Additional serving runtime arguments** in the **Configuration parameters** section:

```
--trust-remote-code
```



NOTE

Only use the **--trust-remote-code** argument with models from trusted sources.

6. Click **Deploy**.

Verification

- If you have configured the vLLM model-serving runtime for speculative decoding, use the following example command to verify API requests to your deployed model:

```
curl -v https://<inference_endpoint_url>:443/v1/chat/completions
-H "Content-Type: application/json"
-H "Authorization: Bearer <token>"
```

- If you have configured the vLLM model-serving runtime for multi-modal inferencing, use the following example command to verify API requests to the vision-language model (VLM) that you have deployed:

```
curl -v https://<inference_endpoint_url>:443/v1/chat/completions
-H "Content-Type: application/json"
-H "Authorization: Bearer <token>"
-d '{"model": "<model_name>",
  "messages":
    [{"role": "<role>",
      "content":
        [{"type": "text", "text": "<text>"
        },
        {"type": "image_url", "image_url": "<image_url_link>"
        }
      ]
    }
  ]
}'
```

2.3. ADDING A CUSTOM MODEL-SERVING RUNTIME

A model-serving runtime adds support for a specified set of model frameworks and the model formats supported by those frameworks. You can use the [preinstalled runtimes](#) that are included with OpenShift AI. You can also add your own custom runtimes if the default runtimes do not meet your needs.

As an administrator, you can use the OpenShift AI interface to add and enable a custom model-serving runtime. You can then choose the custom runtime when you deploy a model on the model serving platform.

**NOTE**

Red Hat does not provide support for custom runtimes. You are responsible for ensuring that you are licensed to use any custom runtimes that you add, and for correctly configuring and maintaining them.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- You have built your custom runtime and added the image to a container image repository such as [Quay](#).

Procedure

1. From the OpenShift AI dashboard, click **Settings → Model resources and operations → Serving runtimes**.
The **Serving runtimes** page opens and shows the model-serving runtimes that are already installed and enabled.
2. To add a custom runtime, choose one of the following options:
 - To start with an existing runtime (for example, **vLLM NVIDIA GPU ServingRuntime for KServe**), click the action menu (**:**) next to the existing runtime and then click **Duplicate**.
 - To add a new custom runtime, click **Add serving runtime**.
3. In the **Select the model serving platforms this runtime supports** list, select **Single-model serving platform**.
4. In the **Select the API protocol this runtime supports** list, select **REST** or **gRPC**.
5. Optional: If you started a new runtime (rather than duplicating an existing one), add your code by choosing one of the following options:
 - **Upload a YAML file**
 - a. Click **Upload files**.
 - b. In the file browser, select a YAML file on your computer.
The embedded YAML editor opens and shows the contents of the file that you uploaded.
 - **Enter YAML code directly in the editor**
 - a. Click **Start from scratch**
 - b. Enter or paste YAML code directly in the embedded editor.

**NOTE**

In many cases, creating a custom runtime will require adding new or custom parameters to the **env** section of the **ServingRuntime** specification.

6. Click **Add**.

The **Serving runtimes** page opens and shows the updated list of runtimes that are installed. Observe that the custom runtime that you added is automatically enabled. The API protocol that you specified when creating the runtime is shown.

7. Optional: To edit your custom runtime, click the action menu (⋮) and select **Edit**.

Verification

- The custom model-serving runtime that you added is shown in an enabled state on the **Serving runtimes** page.

2.4. ADDING A TESTED AND VERIFIED RUNTIME

In addition to preinstalled and custom model-serving runtimes, you can also use Red Hat tested and verified model-serving runtimes to support your requirements. For more information about Red Hat tested and verified runtimes, see [Tested and verified runtimes for Red Hat OpenShift AI](#).

You can use the Red Hat OpenShift AI dashboard to add and enable tested and verified runtimes for the model serving platform. You can then choose the runtime when you deploy a model on the model serving platform.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- If you are deploying the IBM Z Accelerated for NVIDIA Triton Inference Server runtime, you have access to IBM Cloud Container Registry to pull the container image. For more information about obtaining credentials to the IBM Cloud Container Registry, see [Downloading the IBM Z Accelerated for NVIDIA Triton Inference Server container image](#).
- If you are deploying the IBM Power Accelerated Triton Inference Server runtime, you can access the container image from the [Triton Inference Server Quay repository](#).

Procedure

1. From the OpenShift AI dashboard, click **Settings → Model resources and operations → Serving runtimes**.
The **Serving runtimes** page opens and shows the model-serving runtimes that are already installed and enabled.
2. Click **Add serving runtime**.
3. In the **Select the model serving platforms this runtime supports** list, select **Single-model serving platform**.
4. In the **Select the API protocol this runtime supports** list, select **REST** or **gRPC**.
5. Click **Start from scratch**.
6. Follow these steps to add the **IBM Power Accelerated for NVIDIA Triton Inference Server** runtime:
 - a. If you selected the **REST** API protocol, enter or paste the following YAML code directly in the embedded editor.

```
apiVersion: serving.kserve.io/v1alpha1
```

```

kind: ServingRuntime
metadata:
  name: triton-ppc64le-runtime
  annotations:
    openshift.io/display-name: Triton Server ServingRuntime for KServe(ppc64le)
spec:
  supportedModelFormats:
    - name: FIL
      version: "1"
      autoSelect: true
    - name: python
      version: "1"
      autoSelect: true
    - name: onnx
      version: "1"
      autoSelect: true
    - name: pytorch
      version: "1"
      autoSelect: true
  multiModel: false
  containers:
    - command:
      - tritonserver
      - --model-repository=/mnt/models
      name: kserve-container
      image: quay.io/powercloud/tritonserver:latest
      resources:
        requests:
          cpu: 2
          memory: 8Gi
        limits:
          cpu: 2
          memory: 8Gi
      ports:
        - containerPort: 8000

```

7. Follow these steps to add the **IBM Z Accelerated for NVIDIA Triton Inference Server** runtime:
 - a. If you selected the **REST** API protocol, enter or paste the following YAML code directly in the embedded editor.

```

apiVersion: serving.kserve.io/v1alpha1
kind: ServingRuntime
metadata:
  name: ibmz-triton-rest
  labels:
    opendatahub.io/dashboard: "true"
spec:
  containers:
    - name: kserve-container
      command:
        - /bin/sh
        - -c
      args:
        - /opt/tritonserver/bin/tritonserver --model-repository=/mnt/models --http-port=8000 -
          -grpc-port=8001 --metrics-port=8002

```

```

image: icr.io/ibmz/ibmz-accelerated-for-nvidia-triton-inference-server:<version>
securityContext:
  allowPrivilegeEscalation: false
  capabilities:
    drop:
      - ALL
  runAsNonRoot: true
  seccompProfile:
    type: RuntimeDefault
resources:
  limits:
    cpu: "2"
    memory: 4Gi
  requests:
    cpu: "2"
    memory: 4Gi
ports:
  - containerPort: 8000
    protocol: TCP
protocolVersions:
  - v2
  - grpc-v2
supportedModelFormats:
  - name: onnx-mlir
    version: "1"
    autoSelect: true
  - name: snapml
    version: "1"
    autoSelect: true
  - name: pytorch
    version: "1"
    autoSelect: true

```

- b. If you selected the **gRPC** API protocol, enter or paste the following YAML code directly in the embedded editor.

```

apiVersion: serving.kserve.io/v1alpha1
kind: ServingRuntime
metadata:
  name: ibmz-triton-grpc
  labels:
    opendatahub.io/dashboard: "true"
spec:
  containers:
    - name: kserve-container
      command:
        - /bin/sh
        - -c
      args:
        - /opt/tritonserver/bin/tritonserver --model-repository=/mnt/models --grpc-port=8001
        --http-port=8000 --metrics-port=8002
      image: icr.io/ibmz/ibmz-accelerated-for-nvidia-triton-inference-server:<version>
      securityContext:
        allowPrivilegeEscalation: false
        capabilities:
          drop:

```

```

- ALL
runAsNonRoot: true
seccompProfile:
  type: RuntimeDefault
resources:
  limits:
    cpu: "2"
    memory: 4Gi
  requests:
    cpu: "2"
    memory: 4Gi
ports:
- containerPort: 8001
  name: grpc
  protocol: TCP
volumeMounts:
- mountPath: /dev/shm
  name: shm
protocolVersions:
- v2
- grpc-v2
supportedModelFormats:
- name: onnx-mlir
  version: "1"
  autoSelect: true
- name: snapml
  version: "1"
  autoSelect: true
- name: pytorch
  version: "1"
  autoSelect: true
volumes:
- emptyDir: null
  medium: Memory
  sizeLimit: 2Gi
  name: shm

```

8. Follow these steps to add the **NVIDIA Triton Inference Server** runtime:

- a. If you selected the **REST** API protocol, enter or paste the following YAML code directly in the embedded editor.

```

apiVersion: serving.kserve.io/v1alpha1
kind: ServingRuntime
metadata:
  name: triton-kserve-rest
  labels:
    opendatahub.io/dashboard: "true"
spec:
  annotations:
    prometheus.kserve.io/path: /metrics
    prometheus.kserve.io/port: "8002"
  containers:
  - args:
    - tritonserver
    - --model-store=/mnt/models

```

```

- --grpc-port=9000
- --http-port=8080
- --allow-grpc=true
- --allow-http=true
image: nvcr.io/nvidia/tritonserver@sha256:xxxxx
name: kserve-container
resources:
  limits:
    cpu: "1"
    memory: 2Gi
  requests:
    cpu: "1"
    memory: 2Gi
  ports:
    - containerPort: 8080
      protocol: TCP
protocolVersions:
- v2
- grpc-v2
supportedModelFormats:
- autoSelect: true
  name: tensorrt
  version: "8"
- autoSelect: true
  name: tensorflow
  version: "1"
- autoSelect: true
  name: tensorflow
  version: "2"
- autoSelect: true
  name: onnx
  version: "1"
- name: pytorch
  version: "1"
- autoSelect: true
  name: triton
  version: "2"
- autoSelect: true
  name: xgboost
  version: "1"
- autoSelect: true
  name: python
  version: "1"

```

- b. If you selected the **gRPC** API protocol, enter or paste the following YAML code directly in the embedded editor.

```

apiVersion: serving.kserve.io/v1alpha1
kind: ServingRuntime
metadata:
  name: triton-kserve-grpc
labels:
  opendatahub.io/dashboard: "true"
spec:
  annotations:
    prometheus.kserve.io/path: /metrics

```

```
prometheus.kserve.io/port: "8002"
containers:
  - args:
    - tritonserver
    - --model-store=/mnt/models
    - --grpc-port=9000
    - --http-port=8080
    - --allow-grpc=true
    - --allow-http=true
  image: nvcr.io/nvidia/tritonserver@sha256:xxxxx
  name: kserve-container
  ports:
    - containerPort: 9000
      name: h2c
      protocol: TCP
  volumeMounts:
    - mountPath: /dev/shm
      name: shm
  resources:
    limits:
      cpu: "1"
      memory: 2Gi
    requests:
      cpu: "1"
      memory: 2Gi
  protocolVersions:
    - v2
    - grpc-v2
  supportedModelFormats:
    - autoSelect: true
      name: tensorrt
      version: "8"
    - autoSelect: true
      name: tensorflow
      version: "1"
    - autoSelect: true
      name: tensorflow
      version: "2"
    - autoSelect: true
      name: onnx
      version: "1"
    - name: pytorch
      version: "1"
    - autoSelect: true
      name: triton
      version: "2"
    - autoSelect: true
      name: xgboost
      version: "1"
    - autoSelect: true
      name: python
      version: "1"
  volumes:
    - name: shm
```

```
emptyDir: null
medium: Memory
sizeLimit: 2Gi
```

9. Follow these steps to add the **Seldon MLServer** runtime:

- a. If you selected the **REST** API protocol, enter or paste the following YAML code directly in the embedded editor.

```
apiVersion: serving.kserve.io/v1alpha1
kind: ServingRuntime
metadata:
  name: mlserver-kserve-rest
  labels:
    opendatahub.io/dashboard: "true"
spec:
  annotations:
    openshift.io/display-name: Seldon MLServer
    prometheus.kserve.io/port: "8080"
    prometheus.kserve.io/path: /metrics
  containers:
    - name: kserve-container
      image:
        'docker.io/seldonio/mlserver@sha256:07890828601515d48c0fb73842aaf197cbcf245a5c855c789e890282b15ce390'
      env:
        - name: MLSERVER_HTTP_PORT
          value: "8080"
        - name: MLSERVER_GRPC_PORT
          value: "9000"
        - name: MODELS_DIR
          value: /mnt/models
      resources:
        requests:
          cpu: "1"
          memory: 2Gi
        limits:
          cpu: "1"
          memory: 2Gi
      ports:
        - containerPort: 8080
          protocol: TCP
      securityContext:
        allowPrivilegeEscalation: false
        capabilities:
          drop:
            - ALL
        privileged: false
        runAsNonRoot: true
  protocolVersions:
    - v2
  multiModel: false
  supportedModelFormats:
    - name: sklearn
      version: "0"
      autoSelect: true
```

```

    priority: 2
  - name: sklearn
    version: "1"
    autoSelect: true
    priority: 2
  - name: xgboost
    version: "1"
    autoSelect: true
    priority: 2
  - name: xgboost
    version: "2"
    autoSelect: true
    priority: 2
  - name: lightgbm
    version: "3"
    autoSelect: true
    priority: 2
  - name: lightgbm
    version: "4"
    autoSelect: true
    priority: 2
  - name: mlflow
    version: "1"
    autoSelect: true
    priority: 1
  - name: mlflow
    version: "2"
    autoSelect: true
    priority: 1
  - name: catboost
    version: "1"
    autoSelect: true
    priority: 1
  - name: huggingface
    version: "1"
    autoSelect: true
    priority: 1

```

- b. If you selected the **gRPC** API protocol, enter or paste the following YAML code directly in the embedded editor.

```

apiVersion: serving.kserve.io/v1alpha1
kind: ServingRuntime
metadata:
  name: mlserver-kserve-grpc
  labels:
    opendatahub.io/dashboard: "true"
spec:
  annotations:
    openshift.io/display-name: Seldon MLServer
    prometheus.kserve.io/port: "8080"
    prometheus.kserve.io/path: /metrics
  containers:
    - name: kserve-container
      image:
        'docker.io/seldonio/mlserver@sha256:07890828601515d48c0fb73842aaf197cbcf245a5c85

```



```

5c789e890282b15ce390'
env:
  - name: MLSERVER_HTTP_PORT
    value: "8080"
  - name: MLSERVER_GRPC_PORT
    value: "9000"
  - name: MODELS_DIR
    value: /mnt/models
resources:
  requests:
    cpu: "1"
    memory: 2Gi
  limits:
    cpu: "1"
    memory: 2Gi
ports:
  - containerPort: 9000
    name: h2c
    protocol: TCP
securityContext:
  allowPrivilegeEscalation: false
capabilities:
  drop:
    - ALL
  privileged: false
  runAsNonRoot: true
protocolVersions:
  - v2
multiModel: false
supportedModelFormats:
  - name: sklearn
    version: "0"
    autoSelect: true
    priority: 2
  - name: sklearn
    version: "1"
    autoSelect: true
    priority: 2
  - name: xgboost
    version: "1"
    autoSelect: true
    priority: 2
  - name: xgboost
    version: "2"
    autoSelect: true
    priority: 2
  - name: lightgbm
    version: "3"
    autoSelect: true
    priority: 2
  - name: lightgbm
    version: "4"
    autoSelect: true
    priority: 2
  - name: mlflow
    version: "1"

```

```

    autoSelect: true
    priority: 1
  - name: mlflow
    version: "2"
    autoSelect: true
    priority: 1
  - name: catboost
    version: "1"
    autoSelect: true
    priority: 1
  - name: huggingface
    version: "1"
    autoSelect: true
    priority: 1

```

10. In the **metadata.name** field, make sure that the value of the runtime you are adding does not match a runtime that you have already added.
11. Optional: To use a custom display name for the runtime that you are adding, add a **metadata.annotations.openshift.io/display-name** field and specify a value, as shown in the following example:

```

apiVersion: serving.kserve.io/v1alpha1
kind: ServingRuntime
metadata:
  name: kserve-triton
  annotations:
    openshift.io/display-name: Triton ServingRuntime

```



NOTE

If you do not configure a custom display name for your runtime, OpenShift AI shows the value of the **metadata.name** field.

12. Click **Create**.
The **Serving runtimes** page opens and shows the updated list of runtimes that are installed. Observe that the runtime that you added is automatically enabled. The API protocol that you specified when creating the runtime is shown.
13. Optional: To edit the runtime, click the action menu (⋮) and select **Edit**.

Verification

- The model-serving runtime that you added is shown in an enabled state on the **Serving runtimes** page.

Additional resources

- [Tested and verified model-serving runtimes](#)

CHAPTER 3. CONFIGURING MODEL SERVERS ON THE NVIDIA NIM MODEL SERVING PLATFORM

You configure and create a model server on the NVIDIA NIM model serving platform when you deploy an NVIDIA-optimized model. During the deployment process, you select a specific NIM from the available list and configure its properties, such as the number of replicas, server size, and the hardware profile.

3.1. ENABLING THE NVIDIA NIM MODEL SERVING PLATFORM

As an OpenShift AI administrator, you can use the Red Hat OpenShift AI dashboard to enable the NVIDIA NIM model serving platform.



NOTE

If you previously enabled the **NVIDIA NIM model serving platform** in OpenShift AI, and then upgraded to a newer version, re-enter your NVIDIA personal API key to re-enable the NVIDIA NIM model serving platform.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- You have enabled the model serving platform. You do not need to enable a preinstalled runtime. For more information about enabling the model serving platform, see [Enabling the model serving platform](#).
- The **disableNIMModelServing** dashboard configuration option is set to **false**. For more information about setting dashboard configuration options, see [Customizing the dashboard](#).
- You have enabled GPU support in OpenShift AI. This includes installing the Node Feature Discovery Operator and NVIDIA GPU Operator. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).
- You have an NVIDIA Cloud Account (NCA) and can access the NVIDIA GPU Cloud (NGC) portal. For more information, see [NVIDIA GPU Cloud user guide](#).
- Your NCA account is associated with the NVIDIA AI Enterprise Viewer role.
- You have generated a personal API key on the NGC portal. For more information, see [Generating a Personal API Key](#).

Procedure

1. In the left menu of the OpenShift AI dashboard, click **Applications → Explore**.
2. On the **Explore** page, find the **NVIDIA NIM** tile.
3. Click **Enable** on the application tile.
4. Enter your personal API key and then click **Submit**.

Verification

- The NVIDIA NIM application that you enabled is displayed on the **Enabled** page.

CHAPTER 4. CUSTOMIZING MODEL DEPLOYMENTS

You can customize a model's deployment to suit your specific needs, for example, to deploy a particular family of models or to enhance an existing deployment. You can modify the runtime configuration for a specific deployment by setting additional serving runtime arguments and environment variables.

These customizations apply only to the selected model deployment and do not change the default runtime configuration. You can set these parameters when you first deploy a model or by editing an existing deployment.

4.1. CUSTOMIZING THE PARAMETERS OF A DEPLOYED MODEL-SERVING RUNTIME

You might need additional parameters beyond the default ones to deploy specific models or to enhance an existing model deployment. In such cases, you can modify the parameters of an existing runtime to suit your deployment needs.



NOTE

Customizing the parameters of a runtime only affects the selected model deployment.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- You have deployed a model.

Procedure

1. From the OpenShift AI dashboard, click **AI hub** → **Deployments**.
The **Deployments** page opens.
2. Click **Stop** next to the name of the model you want to customize.
3. Click the action menu () and select **Edit**.
The **Configuration parameters** section shows predefined serving runtime parameters, if any are available.
4. Customize the runtime parameters in the **Configuration parameters** section:
 - a. Modify the values in **Additional serving runtime arguments** to define how the deployed model behaves.
 - b. Modify the values in **Additional environment variables** to define variables in the model's environment.



NOTE

Do not modify the port or model serving runtime arguments, because they require specific values to be set. Overwriting these parameters can cause the deployment to fail.

5. After you are done customizing the runtime parameters, click **Redeploy** to save.

- Click **Start** to deploy the model with your changes.

Verification

- Confirm that the deployed model is shown on the **Deployments** tab for the project, and on the **Deployments** page of the dashboard with a checkmark in the **Status** column.
- Confirm that the arguments and variables that you set appear in **spec.predictor.model.args** and **spec.predictor.model.env** by one of the following methods:
 - Checking the InferenceService YAML from the OpenShift Console.
 - Using the following command in the OpenShift CLI:

```
oc get -o json inferencservice <inferencservicename/modelname> -n <projectname>
```

4.2. CUSTOMIZABLE MODEL SERVING RUNTIME PARAMETERS

You can modify the parameters of an existing model serving runtime to suit your deployment needs.

For more information about parameters for each of the supported serving runtimes, see the following table:

Serving runtime	Resource
NVIDIA Triton Inference Server	NVIDIA Triton Inference Server: Model Parameters
OpenVINO Model Server	OpenVINO Model Server Features: Dynamic Input Parameters
Seldon MLServer	MLServer Documentation: Model Settings
vLLM NVIDIA GPU ServingRuntime for KServe	vLLM: Engine Arguments OpenAI-Compatible Server
vLLM AMD GPU ServingRuntime for KServe	vLLM: Engine Argument OpenAI-Compatible Server
vLLM Intel Gaudi Accelerator ServingRuntime for KServe	vLLM: Engine Arguments OpenAI-Compatible Server

Additional resources

- [Customizing the parameters of a deployed model serving runtime](#)

4.3. CUSTOMIZING THE VLLM MODEL-SERVING RUNTIME

In certain cases, you may need to add additional flags or environment variables to the **vLLM ServingRuntime for KServe** runtime to deploy a family of LLMs.

The following procedure describes customizing the vLLM model-serving runtime to deploy a Llama, Granite or Mistral model.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- For Llama model deployment, you have downloaded a meta-llama-3 model to your object storage.
- For Granite model deployment, you have downloaded a granite-7b-instruct or granite-20B-code-instruct model to your object storage.
- For Mistral model deployment, you have downloaded a mistral-7B-Instruct-v0.3 model to your object storage.
- You have enabled the **vLLM ServingRuntime for KServer** runtime.
- You have enabled GPU support in OpenShift AI and have installed and configured the Node Feature Discovery Operator on your cluster. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#)

Procedure

1. Follow the steps to deploy a model as described in [Deploying models on the model serving platform](#).
2. In the **Serving runtime** field, select **vLLM ServingRuntime for KServe**
3. If you are deploying a meta-llama-3 model, add the following arguments under **Additional serving runtime arguments** in the **Configuration parameters** section:

```

--distributed-executor-backend=mp 1
--max-model-len=6144 2

```

1 Sets the backend to multiprocessing for distributed model workers

2 Sets the maximum context length of the model to 6144 tokens

4. If you are deploying a granite-7B-instruct model, add the following arguments under **Additional serving runtime arguments** in the **Configuration parameters** section:

```

--distributed-executor-backend=mp 1

```

1 Sets the backend to multiprocessing for distributed model workers

5. If you are deploying a granite-20B-code-instruct model, add the following arguments under **Additional serving runtime arguments** in the **Configuration parameters** section:

```

--distributed-executor-backend=mp 1
--tensor-parallel-size=4 2
--max-model-len=6448 3

```

- 1 Sets the backend to multiprocessing for distributed model workers
 - 2 Distributes inference across 4 GPUs in a single node
 - 3 Sets the maximum context length of the model to 6448 tokens
6. If you are deploying a mistral-7B-Instruct-v0.3 model, add the following arguments under **Additional serving runtime arguments** in the **Configuration parameters** section:

```
--distributed-executor-backend=mp 1  
--max-model-len=15344 2
```

- 1 Sets the backend to multiprocessing for distributed model workers
 - 2 Sets the maximum context length of the model to 15344 tokens
7. Click **Deploy**.

Verification

- Confirm that the deployed model is shown on the **Deployments** tab for the project, and on the **Deployments** page of the dashboard with a checkmark in the **Status** column.
- For granite models, use the following example command to verify API requests to your deployed model:

```
curl -q -X 'POST' \  
  "https://<inference_endpoint_url>:443/v1/chat/completions" \  
  -H 'accept: application/json' \  
  -H 'Content-Type: application/json' \  
  -d "{  
    \"model\": \"<model_name>\",  
    \"prompt\": \"<prompt>\",  
    \"max_tokens\": <max_tokens>,  
    \"temperature\": <temperature>  
  }"
```